REWT — Presenter reference

CSCI 635 · Kenju Tomita · October 28, 2026 · 10-minute talk + Q&A

Start here — the 20-second version

I built a compact-memory research model with learned energy refinement. Small models show strong synthetic recall and faster decoding. A larger LFM3 conversion has only partially recovered its capabilities. The open question is whether energy refinement improves answers enough to justify its extra computation.

Keep three experiments separate: original joint-energy model; ~112K-parameter streaming comparison; 1.784B-parameter converted LFM3. The small-model speed and recall figures are not large-model results.

If time is short: spend less time on the equation and speed slide. Keep the recall result, conversion result, caveats and final research question. The three appendix slides are for questions only.

Open the presentation

Open the GitHub repository

Numbers at a glance

Capability recovery — same small suite

Check	Before	Recommended stage 2	Original LFM3
Science label ranking	43/128 (33.6%)	59/128 (46.1%)	94/128 (73.4%)
Strict exact responses	9/50 (18%)	13/50 (26%)	25/50 (50%)
Correct code content, 561/1,265 tokens	0/8	0/8	8/8
Tool-call JSON	0/4	0/4	4/4

Small streaming study — three-seed means

Model	Long recall	Story bpb, equal updates	Story bpb, equal CPU
Full Transformer	14.35%	1.6065	1.4641
Window Transformer	21.30%	1.5997	1.4666
Hybrid, no refinement	86.11%	1.5638	1.4828
Hybrid + energy	77.31%	1.5631	1.4926

Warmed small-model decode — 32 tokens after 8K

Device	Previous	Optimized	Speedup
CPU	15.43 ms	8.18 ms	1.89×
Shared GB10 GPU	24.51 ms	1.97 ms	12.44×

Recovery recommendation: stage 2, selected post hoc on this exploratory suite. All headline generation comparisons use greedy decoding, repetition penalty 1.0, the same non-thinking chat template and a 64-new-token cap. Exact-output scores mix answer content and format compliance.

Carried state: small optimized hybrid 37,392 bytes versus full attention 8,421,392 bytes at an 8K prefix (~225×). This excludes weights and workspace. The converted model is separate: 18.783 MiB checked carried state. Memory capacity is not a guarantee of retaining information.

Additional training: stage 1 = 739,862 exposures; stage 2 = 550,381; stage 3 = 373,143. Recommended checkpoint = first two stages, 1,290,243 exposures. Total attempt = 1,663,386. Stage 3 science/exact scores = 58/128 and 9/50.

Slide-by-slide speaking notes

01 · REWT: compact memory and state refinement · 0:00–0:25

Cue: One question: can refinement improve useful memory?

My project is REWT, the Recurrent Energy Wave Transformer. I am studying whether learned energy refinement can improve answers retrieved from compact memory. I have implemented several versions, so I will separate the original energy-based proposal, a small streaming comparison, and the latest conversion of a pretrained model. The results include both useful progress and clear failures.

Transition: First, here is a memory problem we can check directly.

Keep precise: Do not describe the entire 1.784B model as energy minimization.

Evidence: README.md; course_project/PROPOSAL.md

02 · Remember a fact through distractions · 0:25–1:10

Cue: K is the useful fact; X entries are irrelevant activity.

Imagine an assistant reading a continuing stream. It sees K equals eight, then many unrelated X entries, and eventually gets asked for K. A compact memory should retain the useful fact without storing every earlier token. In the saved 64-distractor example, both hybrids answer eight; the full and window Transformers answer two and nine. That is only one recorded example. At 128 distractors the energy hybrid also fails. The aggregate comparison comes later. This is a diagnostic for learned memory, not a task that requires an LLM: a dictionary would solve this grammar exactly.

Transition: The research question is whether an energy score can improve the model’s internal candidate.

Keep precise: The displayed stream is abbreviated. The recorded examples are previously tested cases, not fresh benchmark evidence.

Evidence: course_project/presentation_20261028/streaming_showcase/demo/transcript.txt

03 · Energy is a learned compatibility score · 1:10–1:55

Cue: A learned landscape, not physical energy or truth.

Energy here is a scalar score. Picture the model’s candidate state as a point on this landscape. The gradient gives a direction that lowers the score, and a controlled step size keeps the update stable. The landscape itself is learned during training. That is why a lower valley can still correspond to a wrong answer. The drawing is illustrative and one-dimensional; the actual state has many coordinates. The cosine waves add learned nonlinear preferences around an anchored proposal. This is not sound, a quantum effect, or a claim that the system finds a global minimum.

Transition: There are two architectural versions to keep distinct.

Keep precise: Lower internal energy does not establish correctness, reasoning, or a global optimum.

Evidence: rewt/streaming.py: LatentEnergy; energy_landscape.png

04 · The original joint-energy proposal · 1:55–2:45

Cue: Original: coupled workspace. Current scale-up: backbone plus head.

The original proposal puts recurrent, attention, feedforward and output states into one joint workspace. A scalar energy couples those blocks, with a stabilizing quadratic and learned cosine terms. That implementation remains in the repository. The newer streaming models take a more practical route: conventional memory, attention and feedforward computation propose a hidden state, then a final energy head refines it. The large converted model follows that second design. I am not claiming that its full forward pass is joint energy minimization. It gives me a scalable backbone on which to test whether the refinement earns its cost.

Transition: In either version, training the weights and refining a state are different operations.

Keep precise: Do not transfer the original joint-state descent claim to the whole streaming backbone.

Evidence: rewt/model.py; rewt/streaming.py; rewt/lfm_conversion.py

05 · Training and inference optimize different things · 2:45–3:30

Cue: Training changes weights. Inference changes the candidate state.

During training, examples change the parameters that define the proposals and energy landscape. The small models use supervised next-token losses. The conversion also uses distillation: it matches the pretrained donor’s token distribution while adapting the new operators. Gradients can pass through the refinement steps during training. At inference, the learned weights stay fixed. Only the candidate hidden state is updated using the gradient of energy with respect to that state. The streaming models normally use two refinement rounds. The loop is therefore additional inference computation, not learning new model weights from the user’s prompt.

Transition: That distinction also tells us what to measure.

Keep precise: Recovered model weights changed during offline training; fixed-weight inference is a separate claim.

Evidence: rewt/lfm_distill.py; scripts/recover_lfm3_capabilities.py

06 · Separate numerical behavior, quality and cost · 3:30–4:10

Cue: Three columns: score, answer, cost.

I separate three questions. First, is the numerical implementation correct? That includes gradients, causal behavior and cached decoding. Second, is the answer actually right, including recall and requested formatting? Third, what are the memory and runtime costs? The small streaming comparison has three training seeds and held-out task identities. The larger recovery attempt is one exploratory training trajectory, and I used its small capability suite to recommend the final version. Those are different levels of evidence. A passing numerical test or lower language loss does not certify general capabilities.

Transition: The strongest small-model result is compact recall.

Keep precise: Do not say all studies use three seeds or that the recovery recommendation is an untouched test result.

Evidence: reports/rewt_streaming_20261026/FINDINGS.md; reports/lfm3_capability_recovery_20261028/FINDINGS.md

07 · Small streaming recall results · 4:10–5:15

Cue: 86.1% without energy; 77.3% with energy. Credit memory.

These are small models, around 112 thousand parameters, trained from scratch with matched parameter counts. Across 72 long-recall prompts and three seeds, the full Transformer reaches 14.4 percent and the window Transformer 21.3 percent. The hybrid without refinement reaches 86.1 percent; the energy hybrid reaches 77.3 percent. The hybrid combines gated delta memory with local attention. This supports that compact memory recipe on this synthetic task, but it does not show a recall benefit from energy. Both Transformer controls have a weak third seed, and I used one common tuning recipe, so these are not best-tuned baseline comparisons. The longest recall input is 520 bytes. The later 8K figure is a separate runtime experiment.

Transition: I also reduced the cost of the existing decoder without changing its weights.

Keep precise: These scores are not from the 1.784B converted model. Accuracy is not measured at 8K.

Evidence: reports/rewt_streaming_20261026/FINDINGS.md

08 · Same-weight decoding speed · 5:15–6:00

Cue: Warmed generation only; same outputs and refinement rounds.

For the same small energy-model checkpoint, generating 32 tokens after an 8K prefix takes 8.18 milliseconds instead of 15.43 on CPU, and 1.97 instead of 24.51 on the shared GB10. The changes are fixed-shape decoding, ring caches, compilation and CUDA graph replay. They also remove per-token host transfers. All model controls receive the same execution options. The compact hybrid carries about 225 times less state than full attention in this setup. That excludes parameters and temporary workspace. These are warmed decode measurements, not startup, training or 1.7B-model speed claims. The window Transformer remains faster on CPU.

Transition: The next challenge was transferring pretrained capabilities into this architecture.

Keep precise: The 12.44× GPU gain is not a 12× arithmetic-kernel speedup. Prefill and compilation are excluded.

Evidence: reports/rewt_decode_speed_20261027/FINDINGS.md

09 · LFM3 architecture conversion · 6:00–7:05

Cue: Reuse useful weights; replace operators; recover behavior.

I converted a pinned LFM3-1.7B checkpoint into a standalone 1.784-billion-parameter streaming REWT. I inherited its tokenizer, embeddings, feedforward blocks and compatible attention weights. I replaced the thirteen convolution operators with delta memory, bounded all eleven attention layers to 256-token windows, and added a final two-round energy head. These are substantial changes to information flow, so copying weights does not preserve the donor’s behavior. Recovery matches the donor’s full token distribution, includes next-token and hidden-state losses, and adapts selected feedforward blocks. The resulting model runs from its own PyTorch weights and code; it does not call the donor at inference.

Transition: The measured recovery is useful but still very incomplete.

Keep precise: Bounded attention does not mean all older information is erased: delta memory can carry a compressed state, but retention is imperfect.

Evidence: reports/lfm3_conversion_20261028/DESIGN.md;
reports/lfm3_capability_recovery_20261028/FINDINGS.md

10 · Recovery results and checkpoint choice · 7:05–8:20

Cue: 43 → 59 versus donor 94; exact 9 → 13 versus 25.

On the same 128 science questions, the converted starting model scores 43, the recommended recovered checkpoint scores 59, and the donor scores 94. This is a custom answer-label-ranking protocol, not a published ARC benchmark score. Exact-response probes improve from nine out of fifty to thirteen, versus twenty-five for the donor under the same 64-token greedy protocol. The recommended checkpoint includes 1.29 million additional real-token exposures. A third pass unfroze all feedforward blocks and lowered validation loss further, but its capability scores fell to 58 and nine. I therefore kept stage two. That recommendation is post hoc on this small suite; it needs a fresh confirmation set. Continuation loss improves too, but that alone is not capability recovery.

Transition: The remaining failures are the most important guide for the next experiment.

Keep precise: Total work was 1.66M exposures across three passes; stage 2 contains only 1.29M. Token exposures are not unique tokens.

Evidence: reports/lfm3_capability_recovery_20261028/FINDINGS.md; selection.json

11 · What has and has not recovered · 8:20–9:10

Cue: Long recall and tool formatting still fail.

The converted model still fails all eight longer code-recall probes at 561 and 1,265 prompt tokens, while the donor retrieves those codes. It also fails all four tool-call JSON probes. Most of the exact-response gain is short recall formatting: it now returns the bare requested code. Some separate examples still contain malformed JSON or extra arithmetic text. So the result is partial recovery, not an equivalent replacement for LFM3. The 279-test REWT suite and streaming checks establish implementation behavior; they do not establish intelligence. The compact synthetic-memory result and the large-model capability failures are both part of the evidence.

Transition: My proposed next step targets those failures and tests the energy hypothesis directly.

Keep precise: Strict-output failures combine content and formatting. A mathematically correct answer with extra prose can fail an exact-output probe.

Evidence: reports/lfm3_capability_recovery_20261028/recall_content_diagnostic.json; before.json; after.json

12 · Make refinement earn its cost · 9:10–10:00

Cue: Correction training, realistic memory, fresh equal-cost controls.

The next experiment is to teach refinement to repair deliberately corrupted candidates, rather than assuming lower energy will improve an answer. In parallel, the converted model needs varied long-context recall and structured-output recovery. I would compare zero, one, two and four rounds against both a no-energy model and an equally costly feedforward correction head. Settings should be chosen on validation, then frozen before fresh tests. The contribution so far is an implemented and audited research system with compact-memory gains, faster inference, and a documented partial conversion. The open question remains whether energy refinement improves useful retrieval enough to justify its extra computation.

Transition: Thank you. I have the equations, controls and full results available for questions.

Keep precise: The correction experiment is proposed, not completed. Do not claim a demonstrated general energy advantage.

Evidence: reports/rewt_streaming_20261026/FINDINGS.md; reports/lfm3_capability_recovery_20261028/FINDINGS.md

13 · Appendix A: energy math · backup only

Cue: Quadratic anchor + bounded cosine terms; fixed coefficients within the loop.

Let delta be the displacement from the proposed hidden state. The energy is one half delta-transpose P delta plus a sum of cosine terms. P is positive diagonal. The gradient is P delta minus the sum of amplitude times sine times the frequency vector. A bound on curvature is the largest diagonal precision plus the sum of absolute amplitude times squared frequency norm. The implementation uses step size 0.95 divided by this bound. This supports descent for the fixed smooth energy in exact arithmetic; numerical tests check implementation behavior. It does not guarantee a global optimum or correct answer. The converted energy head computes its updates in FP32.

Transition: The practical question is whether these state changes improve quality enough to pay for them.

Keep precise: Earlier joint-energy solvers use other acceptance mechanisms. This equation and step rule describe the streaming/conversion head.

Evidence: rewt/streaming.py: LatentEnergy; rewt/lfm_conversion.py: ConversionEnergy

14 · Appendix B: controls and negative results · backup only

Cue: Equal CPU story loss favors the Transformer; energy task ablation ties.

The hybrid is not better on every metric. At equal training CPU time, story loss is 1.4641 bits per byte for the full Transformer, 1.4828 for the no-energy hybrid and 1.4926 for the energy hybrid. Lower is better. Disabling refinement in the frozen energy model leaves complete task accuracy at 36.63 percent. A separately trained no-energy model tests the effect on learning, while a frozen intervention tests the inference contribution; they answer different questions. In the large converted checkpoint, energy-off continuation NLL is 2.229599 versus 2.229755 with energy, which also fails to establish a meaningful energy benefit.

Transition: These controls motivate correction training and equal-cost comparisons.

Keep precise: The small-model family scores are dominated by recall; do not label them general reasoning accuracy.

Evidence: reports/rewt_streaming_20261026/FINDINGS.md; reports/lfm3_capability_recovery_20261028/continuation_and_cache.json

15 · Appendix C: sources and project contribution · backup only

Cue: Credit the backbone and donor; own the implementation and controlled analysis.

Energy Transformer motivates energy-based state refinement. Gated Delta Networks is prior work on combining delta memory with local attention. LFM3 supplies the pretrained weights used in conversion. The project does not claim to invent those ingredients. Its contribution is the energy extension, conversion and recovery implementation, optimized execution, and reproducible positive and negative experiments. The repository contains plans, source snapshots, result tables and raw responses. Donor weights retain their source license; large checkpoints and downloaded datasets are not included in the Git repository.

Transition: Point to the relevant report when answering a detailed numerical question.

Keep precise: The current hybrid is not a reproduction of a published Energy Transformer or a proof of a new state of the art.

Evidence: RIGHTS.md; reports/lfm3_conversion_20261028/DESIGN.md; reports/lfm3_capability_recovery_20261028/FINDINGS.md

Demo and online presentation setup

Open the deck in Present mode and keep this reference on a second screen or a separate window. Share only the presentation window. Use the document outline or the HTML navigation to jump to the current slide. Keep the local PDF and the recorded transcript open as offline backups.

The ten-minute talk can use the abbreviated example on slide 2 without switching windows. If you want a live demonstration, rehearse it before the call and use the existing saved transcript if startup or screen sharing takes too long. The first examples at each length are retained, including failures; they were not chosen to make every model succeed.

```
cd /home/kt/Documents/EnergyTransformer
OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 .venv/bin/python
scripts/demo_rewt_streaming.py --output /tmp/rewt_class_demo_live_20261028
```

Use a fresh output directory if that path already exists. The eager CPU path starts without compilation. This script replays three previously published examples for all four small models; it is not a new quality evaluation.

64 distractors: expected code 8; full Transformer returns 2, window Transformer 9, both hybrids 8. 128 distractors: expected code 3; no-energy hybrid returns 3, energy hybrid returns 2. Show that failure if discussing the demonstration.

Recorded demonstration — all inputs and outputs

For the converted model, the verified offline smoke prompt returns exactly “hello”. Avoid making an unrehearsed open-ended answer the main demo: JSON, recall and formatting remain unreliable.

```
cd /home/kt/Documents/EnergyTransformer
.venv-lfm3/bin/python runs/lfm3_capability_recovery_20261028/converted/sample.py --prompt
"Reply with exactly the word hello." --tokens 16
```

Likely questions and ready answers

What is the one-sentence contribution?

I built and evaluated energy refinement on compact recurrent-memory models, optimized their decoding, and tested transfer from a pretrained LFM3 model. The evidence shows synthetic-memory gains and partial recovery, while the accuracy benefit of energy remains unproven.

Is this a completely energy-based Transformer?

The original small joint-workspace implementation is the fully coupled energy proposal. The newer streaming and LFM3-converted models use conventional delta-memory/local-attention/FFN operators followed by a latent-energy head. Keep those versions separate.

What does “wave” mean?

Cosine/Fourier components over the coordinates of the hidden state. They add learned nonlinear preferences to an anchored quadratic energy. This is not a Fourier transform over token positions, an audio signal, or a quantum mechanism.

Why does decreasing energy not guarantee a correct answer?

The model learns the energy function from data. It can assign a low score to an incorrect state, and local descent is not a global search. The score is an internal objective, while task accuracy is an external measurement.

How does delta memory work?

It maintains a compact matrix-like state. A new key/value interaction reads what is already stored and applies a gated residual correction toward the new value. It does not retain an exact copy of all previous tokens. The implementation has sequential and chunked forms with checked algebra and gradients.

Why keep attention too?

Local attention can inspect recent tokens directly, while delta memory carries a compressed longer-lived summary. The roles complement each other. Gated Delta Networks is prior work on this combined backbone; I do not claim that combination as an invention.

Could a dictionary solve the demonstration?

Yes, exactly. That is why it is a diagnostic task rather than an application claim. A practical learned-memory system would need to handle natural-language variation, multiple entities and conflicts, and should be compared with ordinary lookup, retrieval and database baselines.

Where could this be useful?

Possible research directions include local-assistant memory, retaining events in long logs, and remembering clues for game agents. None is demonstrated as a usable product. The current evidence is synthetic associative recall and a limited converted-model evaluation.

Does it beat a Transformer?

Only on some measured outcomes in this small, fixed setup. Long recall improves, but equal-CPU story loss favors the Transformer, the window Transformer decodes faster on CPU, and eager GPU training was slower for the hybrid. The baselines also have an unstable third seed and were not separately tuned to their best settings.

Does energy refinement actually help?

No general accuracy advantage has been established. The separately trained no-energy hybrid is better on long recall. Disabling refinement in the frozen small energy model leaves task accuracy unchanged. Small language-loss shifts do not establish a capability benefit. That negative result motivates the next experiment.

Why is the GPU speedup so large?

The model is small enough that dispatch and synchronization overhead matter. Compilation, CUDA graphs, ring buffers and returning token IDs only at the end remove overhead. It is a warmed end-to-end decode improvement, not a twelvefold improvement in mathematical operations or training speed.

How much memory is saved?

For the optimized small hybrid at an 8K prefix, carried cache/state is 37,392 bytes (36.52 KiB), versus 8,421,392 bytes (8.03 MiB) for full attention: about 225× less. These figures exclude parameters, logits, prepared constants, graph pools and workspace. The converted large model is separate: its checked carried state is 18.783 MiB per sequence after filling its windows.

Is the 8K memory result also an 8K recall result?

No. The small-model recall evaluation ends at 520 input bytes. The 8K experiment measures execution cost with seeded token prompts. The converted model has separate recall probes at 77, 561 and 1,265 prompt tokens; its longer probes still fail.

What was inherited from LFM3?

The tokenizer, token embeddings, feedforward weights, normalization parameters and compatible attention projections. The thirteen convolution operators were replaced by delta-memory modules, eleven attention layers were bounded, and a final energy head was added. The resulting model has 1,784,445,792 parameters.

Was the model trained from scratch?

The small streaming comparison uses cold starts. The 1.784B model is transferred from a pretrained donor and then distilled. These are different experiments. The converted model inherits substantial learned information and cannot be presented as a from-scratch large model.

What exactly is the recovery objective?

Full-vocabulary KL(teacher || student) plus 0.2 times hard next-token loss plus 0.05 times relative hidden-state MSE. Assistant targets have weight 1, other real targets 0.1, and right-padding targets have zero weight. Training uses FP32 trainable weights/optimizer state and BF16 autocast.

How much recovery training was done?

After the initial conversion pilot, this attempt used 1,663,386 real-token exposures across three passes. The recommended second-pass model contains the first 1,290,243 exposures and 1,536 updates. Stage 3 adds 373,143 exposures and 512 updates. These count repeated token presentations, not unique corpus coverage.

Why did you choose stage 2 instead of the latest weights?

Stage 3 improved short-crop validation KL from 0.5024 to 0.4949, but science fell from 59 to 58 out of 128 and exact responses from 13 to 9 out of 50. I retained both and recommended stage 2. That is post-hoc selection on a small suite, not an independent final benchmark.

What does “59 out of 128” measure?

64 ARC-Easy and 64 ARC-Challenge questions under a custom zero-shot protocol. The model ranks the single-token answer labels at the first output position. This is not the standard published ARC/lm-eval protocol and is not a measure of general intelligence.

Why are even donor arithmetic scores low?

The probes require exact output and stop after 64 generated tokens. The donor may emit reasoning or extra prose despite a non-thinking prompt. A correct equation with extra text fails a request for only the number. The table therefore combines answer content with instruction compliance; it must not be read as raw mathematical competence.

Was there data leakage?

Training and validation use hashed conversation partitions; audits found no identical full token sequences or checked 128/512-token prefixes shared with the evaluated validation/test sets. Benchmark prompts and answers were not supplied to training. Semantic overlap and donor pretraining contamination are not ruled out. The small capability suite influenced the final recommendation and is not an untouched confirmation set.

Can you prove cached decoding is correct?

There are unit and gradient tests plus full-width numerical audits. The selected converted model's FP32 chunk-relative error is about 1.41e-6, below 1e-4. BF16 shape-dependent error is about 1.52%, so BF16 is not bitwise chunk parity. The cache remains bounded through the checked 1,024-token extension, and branching preserves the original state.

What does the 279-test result mean?

It is the REWT implementation suite: numerical behavior, gradients, causality, data handling and execution contracts. It is not 279 independent capability benchmarks. The wider repository test suite is recorded separately in the delivery verification.

Does the model need LFM3 at inference?

No. The packaged checkpoint has its own code, tokenizer and weights and passed an offline smoke test from outside the repository. It is a custom PyTorch model, not a Hugging Face AutoModel checkpoint. Training uses the donor; inference does not.

What would count as a successful next experiment?

Choose a correction-training setup using validation, freeze it, and test new semantic identities and wordings. Measure whether refinement repairs more wrong answers than a zero-energy model and an equally costly feedforward correction head. Report accuracy versus latency and training compute, including failures and 0/1/2/4-round results.

What if energy still does not help?

That is a useful result. It would show that this score, state space or supervision is insufficient, rather than justify claiming that lower energy equals better reasoning. The compact-memory and execution contributions can be evaluated independently.

Technical details if pressed

Energy gradient and step bound

$E(\delta) = 0.5\ \delta^{T}P\delta + \Sigma\ a_k \cos(\omega_k^{T}\delta + \varphi_k)$. The candidate is anchor + δ. The gradient is $P\delta - \Sigma\ a_k \sin(\omega_k^{T}\delta + \varphi_k)\omega_k$. The diagonal precision lies between 0.15 and 2; amplitudes use tanh divided by the square root of the number of modes. Frequency norms are bounded below 2. Set $L_{bound} = \max \text{diagonal}(P) + \Sigma\ |a_k|\ ||\omega_k||^2$ and $\eta = 0.95/L_{bound}$. Initialize δ to zero and take the configured number of updates. This describes the streaming/conversion head, not every earlier solver.

Recovery data and trainable parameters

The recovery corpus contains 16,100 training conversations and 9,797,398 stored tokens, including 8,931 conversations shorter than 513 tokens; validation has 263 conversations. Data is from a pinned SmolTalk shard. Documents do not share training rows. Stages 1 and 2 train the new mixers, norms, energy parameters and the last six feedforward blocks (672,881,504 trainable parameters). Stage 3 unfreezes all 24 feedforward blocks (1,522,301,792 trainable parameters). Token embeddings remain frozen. The selected stage-2 audit preserves 1,111,564,288 frozen parameters exactly.

Validation and continuation metrics

On the same 512-token validation crop setup, KL is 0.6156 before recovery and 0.5024 for recommended stage 2; stage 3 reaches 0.4949. Stage 2’s own longer-crop training validation changes from 0.6813 to 0.6581 and is not directly comparable to the short-crop values. On 8,192 previously inspected test token positions, continuation NLL changes 2.3468 → 2.2298 (donor 2.3117) and donor KL 0.8280 → 0.7557. A student can have lower NLL on this subset while still losing badly on capabilities.

Earlier structured reasoning result, if someone remembers the old deck

The separate ~68K-parameter repeated-read head predicts structured rule traces from raw bytes. Both repeated-read arms, with and without energy, solved all 960 main test prompts and 480 extra-distractor prompts in each of three seeds. Fixed-query control averaged 27.4%. Different withheld validation phrasings were much weaker (~43.15%). This is a specialized supervised lookup/trace system, not the streaming language model or LFM3 conversion, and the perfect score does not show an energy advantage.

Sources and reproducibility

Small streaming comparison

Decoder timing and scope

Conversion design and weight mapping

Capability recovery, raw answers and selection caveats

Final recovery implementation verification

Earlier repeated-read reasoning study

Energy Transformer

Gated Delta Networks

Pinned LFM3 donor model card

Original checkpoints, the stage-3 alternative and recorded failures remain available locally. GitHub contains source and research evidence; model checkpoints, downloaded corpora and virtual environments remain excluded by the existing repository policy. Follow the donor license for transferred weights.